CSI101

Processors

Lecture 2 — Computer System Components

Dr. Kok Hoe WONG



Overview

1. **What does the CPU actually do?**
   Its job, and the limits of that job

2. **Inside the CPU**
   Control unit, ALU, clock, buses, registers

3. **The fetch–decode–execute cycle**
   How one instruction is actually carried out

4. **What makes one processor faster?**
   Clock speed, cores, cache, width, pipelining

5. **Modern processors**
   Multi-core, SMT, SoC, RISC and CISC

6. **Data representation**
   Bits, bytes, binary, hex, and negative numbers

Where we left off

Last lecture ended with the von Neumann model. This lecture opens the box marked ‘processor’.

One memory, two contents

Program and data share a single rewriteable store, both as binary.

The processor is the active part

It fetches instructions from memory and executes them in order.

The bottleneck

Instructions and data cross the same path — the CPU waits.

No standard instruction set

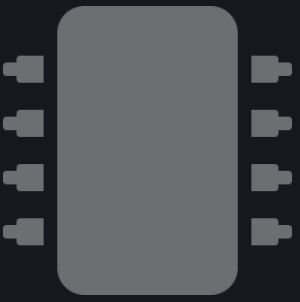
Each CPU has its own fixed set of 100-200 instructions.

Today: what is inside that active part, and how it turns binary patterns into work.

1

What does the CPU actually do?

The job, and the surprisingly narrow limits of that job.



The CPU’s job, in one sentence

The central processing unit repeatedly fetches an instruction from memory, works out what it means, and carries it out — billions of times per second.

WHAT IT CAN DO

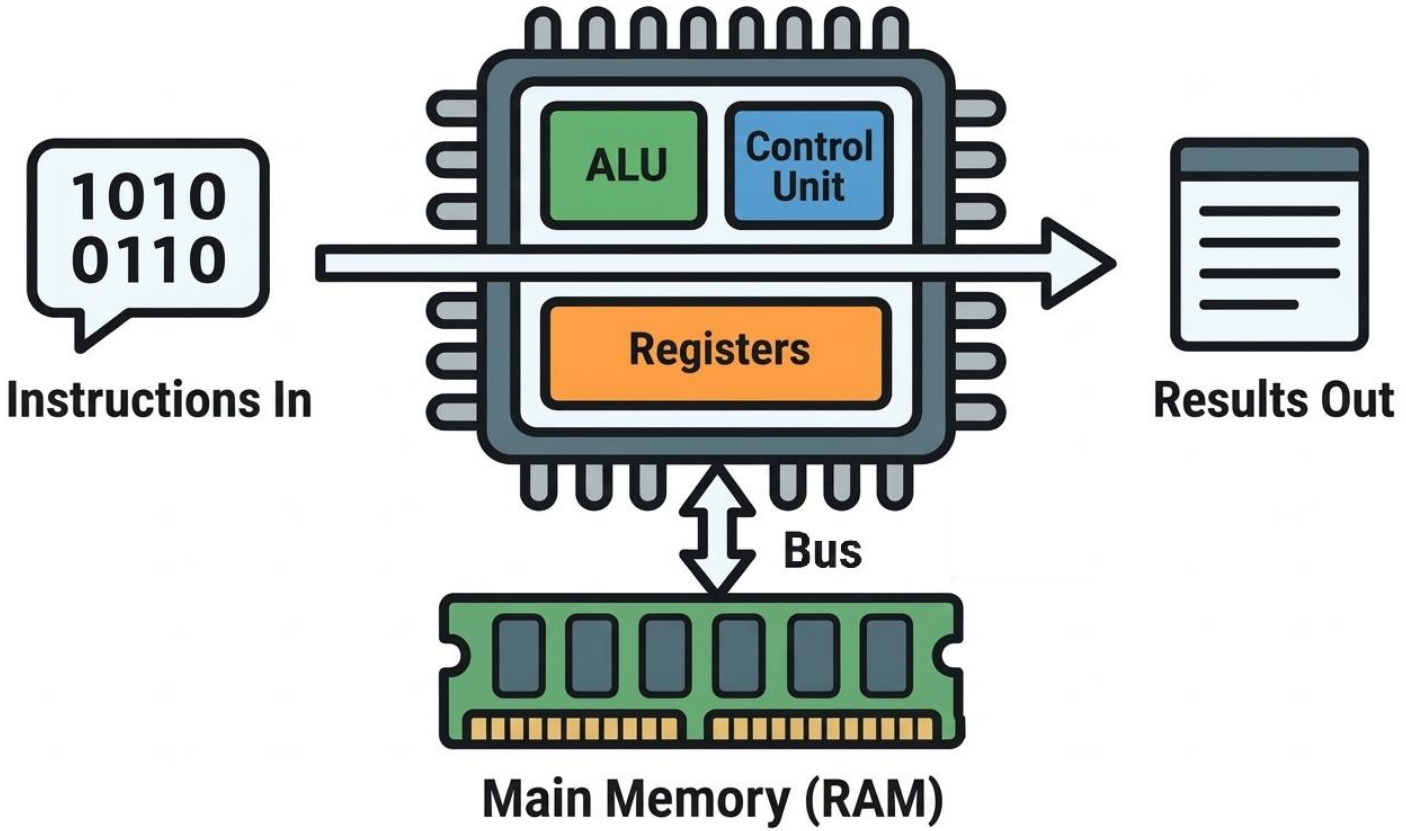
- Move a value between memory and a register
- Add, subtract, multiply, divide
- Compare two values: equal, less than, greater than
- Perform bitwise logic: AND, OR, NOT, XOR
- Change which instruction runs next

WHAT IT CANNOT DO

- Understand a word, an image or a sound
- Read from a disk or a network on its own
- Do anything it has not been given an instruction for
- Know whether the bits it just added were a price or a pixel

Everything a computer appears to do is built from the list on the left.

Where the CPU sits



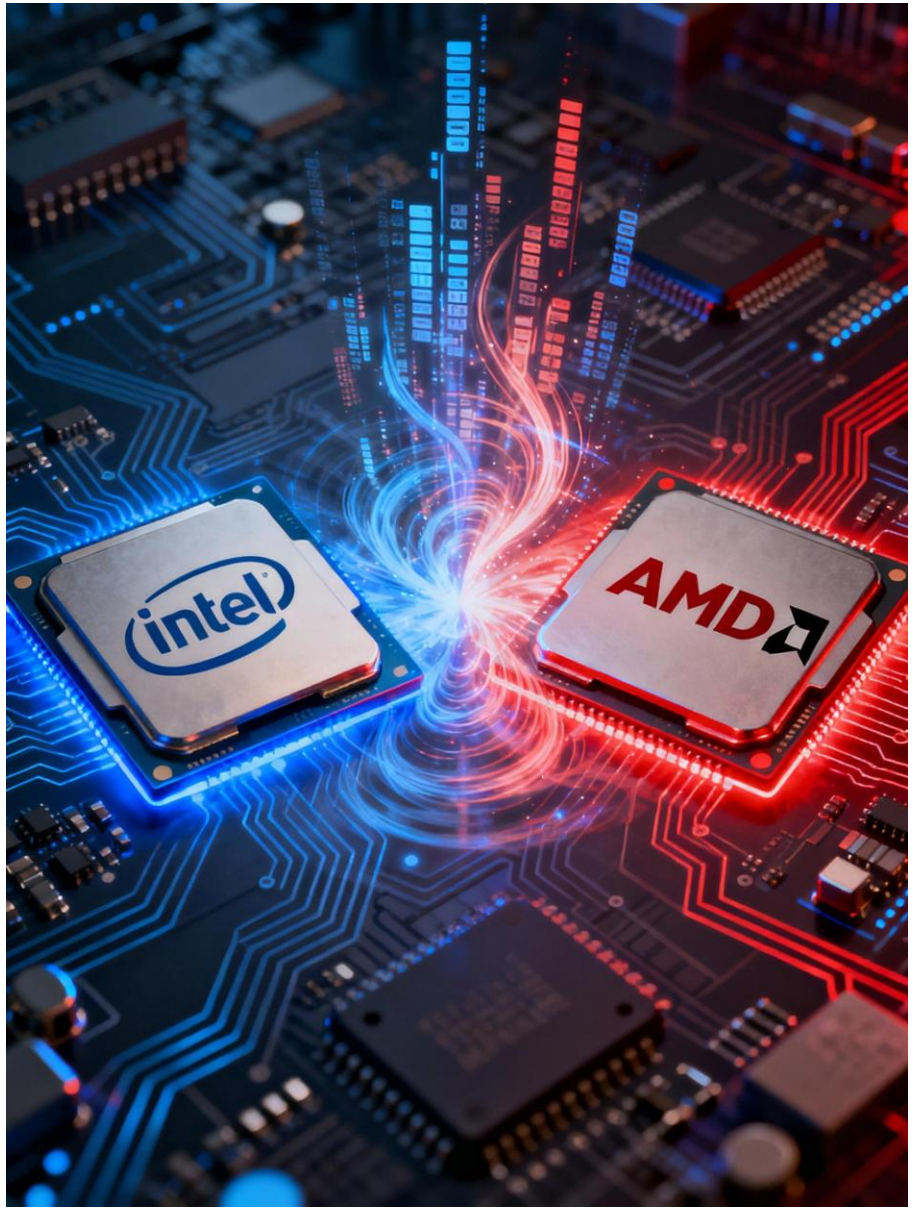
The CPU never touches a keyboard or a disk directly. Everything reaches it through memory, over the bus.

What an instruction looks like

- Every instruction is a binary pattern held in memory
- It splits into an opcode — what to do — and one or more operands — what to do it to
- The CPU's instruction set is the complete list of patterns it recognises
- That set is fixed in the silicon and cannot be extended
- Assembly language gives each pattern a name humans can read

```
ADD   R1, R2
```

opcode: ADD operands: R1, R2



2

Inside the CPU

Control unit, ALU, clock, buses and registers — and what each one is for.



The five key components



Control unit

Understands instructions and directs everything else



ALU

Performs the arithmetic and the logic



Clock

Sets the rhythm and synchronises the components



Buses

Carry addresses, data and control signals



Registers

Very small, very fast storage inside the CPU

Only the registers store anything. The other four move, decide, calculate or synchronise.

The control unit

The control unit runs the show.

- It understands each instruction and tells the other components what that instruction needs from them
- It fetches instructions from memory in the right order
- It decodes the opcode to work out which operation is required
- It sends control signals along the control bus at the right moment
- It moves data between memory, registers and the ALU



CHARACTERISTICS

Organised, efficient, collaborative

IT MANAGES

The instructions — and the other components that carry them out.

The arithmetic logic unit

The ALU is the calculator of the CPU.

ARITHMETIC OPERATIONS

- Addition, subtraction, multiplication, division
- Increment and decrement

LOGICAL OPERATIONS

- Compare two values — numbers, letters or symbols
- Test for one of three conditions: equal to, less than, greater than
- Bitwise logic: AND, OR, NOT, XOR



CHARACTERISTICS

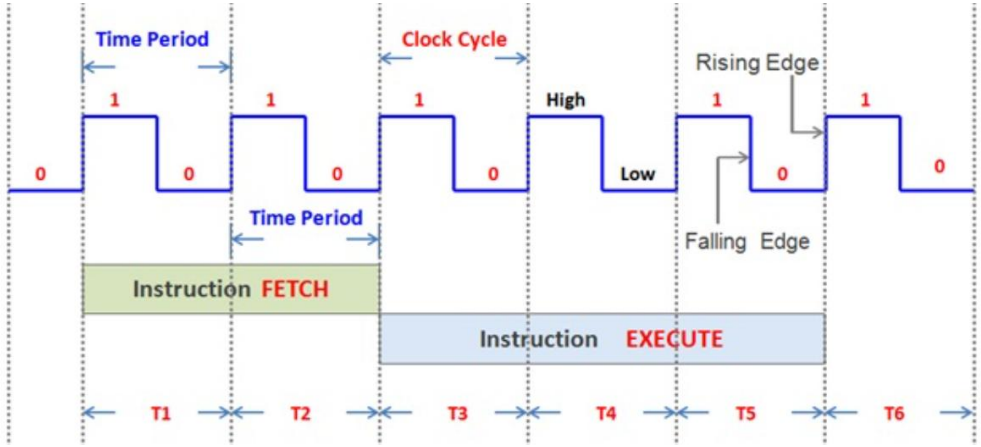
Logical, good with numbers, precise

IT MANAGES

Calculations and logic. Results usually land in the “accumulator” — in older/simpler processor architectures, it is just a register with a special role.

The clock

The CPU contains an internal clock that regulates the number of cycles carried out per second and synchronises the other components.



Cycles per second

Measured in hertz. 1 GHz is one billion clock cycles every second.

It synchronises

Components act on the tick, so signals arrive in a known order.

Not one instruction per tick

A simple instruction may take several cycles; a modern CPU may finish several per cycle.

Punctual, reliable, good rhythm — the clock manages the cycles per second.

Buses: the wires between everything

A bus is a shared set of wires connecting subsystems. Three run between the CPU and memory.

Address bus

One way: CPU → memory

Carries the address of the location the CPU wants. Its width sets how much memory can be addressed.

Data bus

Two way

Carries the actual value being read or written. Its width sets how many bits move at once.

Control bus

Two way

Carries the signals that say read, write, interrupt, or clock tick.

WHY BUS WIDTH MATTERS

A 32-bit address bus can address 2^{32} locations — about 4 GB. A 64-bit address bus removes that ceiling entirely. This is the real reason 32-bit machines could not use more than 4 GB of RAM.

Registers

Registers are very small, very fast memory locations located inside the CPU — and they work under the direction of the control unit.

MAR

Memory address register

Stores the memory address the CPU is looking at

MDR

Memory data register

Stores the data once it has been fetched from memory

CIR

Current instruction register

Holds the binary representation of the instruction being executed

PC

Program counter

Counts up as each instruction is executed, tracking position in the program

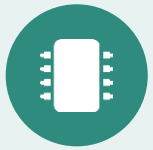
Acc

Accumulator

Stores important data being used in calculations

All five are fast, reliable memory — and all five appear in the fetch-decode-execute cycle.

Registers are not memory



Registers

- Located inside the CPU
- A handful of them — tens, not millions
- Hold a single value each, typically 32 or 64 bits
- Accessed in under one nanosecond
- Accept, hold and transfer instructions or data



Main memory (RAM)

- Not part of the CPU — it sits on the motherboard
- Billions of locations
- Holds whole programs and their data
- Accessed in 50-100 nanoseconds, across the bus
- Keeps information only while the program is running

A register access is roughly a hundred times faster than a memory access. That gap shapes everything.

Check your understanding

Match each description to the component. Answers on the next slide.

A It handles mathematical and logical operations required as part of an instruction.

B It understands the instructions and tells the other components what each instruction needs from them.

C It regulates the number of cycles carried out per second and synchronises the other components.

D They are very small, very fast memory locations located inside the CPU.

E It stores the data once it has been fetched from memory.

F It counts up as each instruction is executed.

Options: control unit · ALU · clock · registers · MDR · program counter

Check your understanding: answers

A **Arithmetic logic unit**

Handles the mathematics and the logic.

B **Control unit**

Understands instructions and directs the other components.

C **Clock**

Regulates cycles per second and synchronises everything.

D **Registers**

Very small, very fast storage inside the CPU.

E **Memory data register (MDR)**

Holds the data once fetched from memory.

F **Program counter (PC)**

Counts up as each instruction is executed.

3

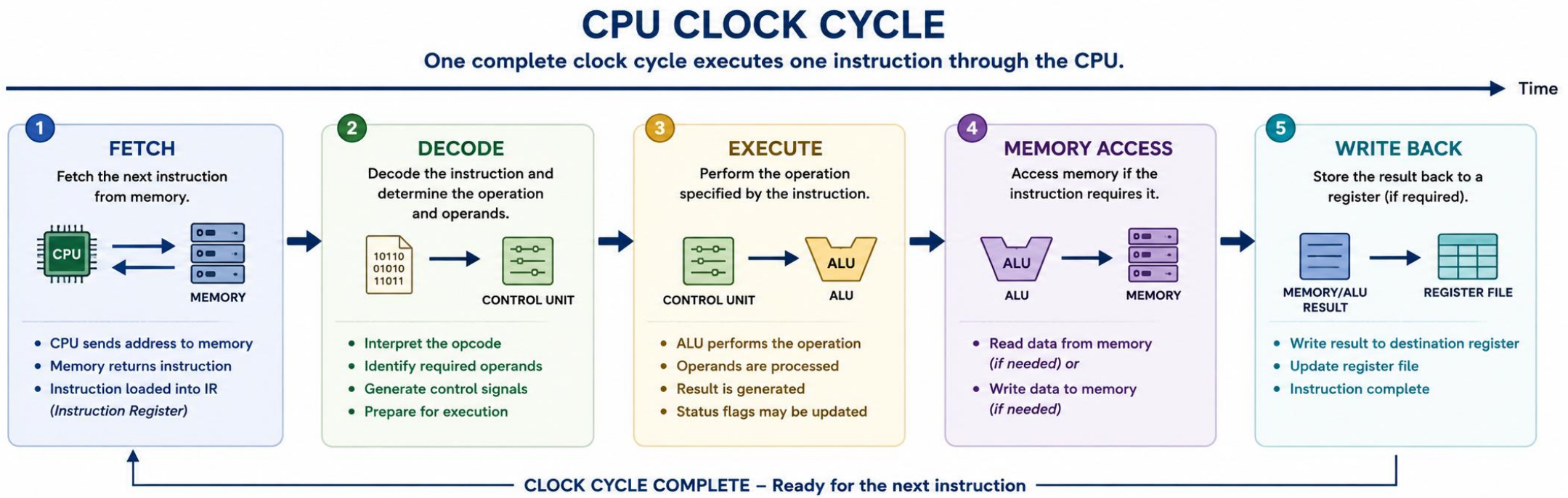
The fetch-decode-execute cycle



How a single instruction actually gets carried out.

The cycle, in outline

Also called the machine cycle. The CPU repeats it for every instruction, for as long as it is powered on.



INSTRUCTION TIME (I-TIME)

Fetch and decode — getting the instruction and understanding it.

EXECUTION TIME (E-TIME)

Execute and store — performing the action and writing the result.

Step 1 — Fetch

Getting the next instruction out of memory, using four registers and two buses.

1. The address in the program counter (PC) is copied into the memory address register (MAR)
2. That address travels to memory along the address bus
3. Memory returns the contents of that location along the data bus into the memory data register (MDR)
4. The instruction is copied from the MDR into the current instruction register (CIR)
5. The program counter is incremented, so it now points at the next instruction

REGISTERS USED

PC	where we are
MAR	which address
MDR	what came back
CIR	the instruction

Why increment the PC now, before the instruction has even run?

Step 2 — Decode

- The control unit examines the instruction now sitting in the CIR
- It splits the binary pattern into the opcode and the operand
- The opcode says which operation is wanted; the operand says what to apply it to
- The control unit works out which components will be needed to carry it out
- It prepares the control signals that will drive those components in the next step

No calculation happens yet. Decode is purely understanding the instruction.

ONE INSTRUCTION, SPLIT APART

```
0001  0110 1100
```

opcode operand

0001 means LOAD on this machine.

0110 1100 is memory address 108.

So: load the value held at address 108 into the accumulator.

Steps 3 and 4 — Execute and store



Execute

- The control unit issues the signals it prepared
- If arithmetic or logic is needed, the ALU does it
- If data must be moved, it moves along the data bus
- If the instruction is a jump, the program counter is overwritten with a new address



Store (write-back)

- The result is written back — usually into the accumulator
- It may instead be written out to a memory location
- Status flags are updated: was the result zero, negative, did it overflow?
- The cycle begins again with the address now in the program counter

Those status flags are how the next instruction will know whether to jump. That is how an if statement works.

Worked example: adding two numbers

A three-instruction program starting at address 0. Memory holds 12 at address 100 and 30 at address 101.

THE PROGRAM

```
0:  LOAD  100
1:  ADD   101
2:  STORE 102
```

ACCUMULATOR

`empty → 12 → 42`

Three instructions. Six trips round the cycle.

PC = 0	Fetch LOAD 100 into CIR. PC becomes 1.
Execute	Value 12 is fetched from address 100 into the accumulator.
PC = 1	Fetch ADD 101 into CIR. PC becomes 2.
Execute	ALU adds the value at address 101 to the accumulator: 12 + 30 = 42.
PC = 2	Fetch STORE 102 into CIR. PC becomes 3.
Execute	The accumulator's 42 is written into memory address 102.

What “3 GHz” actually means

3 GHz = 3,000,000,000 clock cycles every second

One cycle lasts about a third of a nanosecond — in that time, light travels only about 10 cm.

Not one instruction per cycle

A simple instruction may need several cycles. A modern processor may also complete several instructions per cycle.

Cycles are the unit of timing

Everything inside the CPU is measured in cycles, not seconds. Memory access might cost 200 cycles.

Speed is not the whole story

A 3 GHz processor is not automatically faster than a 2.5 GHz one — as the next section explains.

A memory access that costs 200 cycles is 200 instructions the CPU did not get to run.

4

What makes a processor faster?



Six factors — and why the number on the box is the least useful one.

Six things that matter

Clock speed

How many cycles per second

Number of cores

How many instruction streams at once

Cache size

How often the CPU avoids waiting for RAM

Word and bus width

How many bits move and are processed at once

Pipelining

How many instructions are in flight together

Instruction set design

How much work each instruction achieves

Clock speed and cores



Clock speed

- More cycles per second means more work per second — all else being equal
- But all else is rarely equal: two processors at the same speed can differ hugely
- Higher speed means more heat, and heat is the practical ceiling
- This is why clock speeds have sat around 3-5 GHz for two decades



Cores

- A core is a complete processor: its own control unit, ALU and registers
- Four cores can run four instruction streams genuinely at the same time
- But only if the software is written to split the work up
- A single-threaded program gets no benefit at all from the other three cores

Four cores never means four times faster. It means at most four times, and usually far less.

Cache

A small amount of very fast memory on the processor itself, holding copies of what was recently used.

L1	L2	L3	RAM
per core	per core	shared	off-chip
32-64 KB	**256 KB - 2 MB**	**8-64 MB**	**8-64 GB**
~4 cycles	~12 cycles	~40 cycles	~200 cycles

WHY IT WORKS

- Programs reuse the same data and instructions repeatedly
- And they tend to use data stored close together
- So, a copy kept nearby is usually the copy wanted next

WHY IT MATTERS SO MUCH

A value found in L1 costs 4 cycles. The same value fetched from RAM costs around 200. Cache is often worth more to real performance than an extra gigahertz.

L1, L2 and L3 Cache and Why Processors Need Ultra-Fast Memory

Word size, bus width and pipelining

WORD SIZE AND BUS WIDTH

- The word size is how many bits the CPU handles in one operation — today, 64
- A wider data bus moves more bits per transfer
- A wider address bus can reach more memory

INSTRUCTION SET DESIGN

- Some designs do more per instruction; others do less but faster
- What matters is instructions completed per cycle, not instructions available
- We return to this with RISC and CISC shortly

1 Pipelining in Action

With **5 instructions**, a new instruction enters the pipeline every cycle, and after the pipeline is filled, multiple instructions are completed in parallel (one per cycle).

Clock cycle	1	2	3	4	5	6	7	8	9
Instruction 1	IF	ID	EX	MEM	WB				
Instruction 2		IF	ID	EX	MEM	WB			
Instruction 3			IF	ID	EX	MEM	WB		
Instruction 4				IF	ID	EX	MEM	WB	
Instruction 5					IF	ID	EX	MEM	WB

In this example, the pipeline is fully occupied by cycle 5 (all 5 stages are busy).
From cycle 5 onward, one instruction completes every cycle (in the WB stage).

2 Why Pipelining Helps

Higher throughput
Multiple instructions are in progress at the same time, so more work is done per clock cycle.

Better resource utilisation
Each stage is busy most of the time, reducing idle time.

Improved performance
For long instruction sequences, pipelining significantly increases instructions per second (IPS).

Note: Pipelining doesn't make a single instruction faster — it makes the overall throughput higher.

So which processor is faster?

The honest answer: faster at what? A processor that wins at video encoding may lose at running a database.

Compare like with like

Clock speed only compares meaningfully between processors of the same design and generation.

Use benchmarks, carefully

A benchmark measures one workload. Choose one that resembles what you will actually run.

Performance per watt

In phones, laptops and data centres, work done per unit of energy often matters more than raw speed.

Find the real bottleneck

Most systems are limited by memory, storage or the network long before the processor runs out.

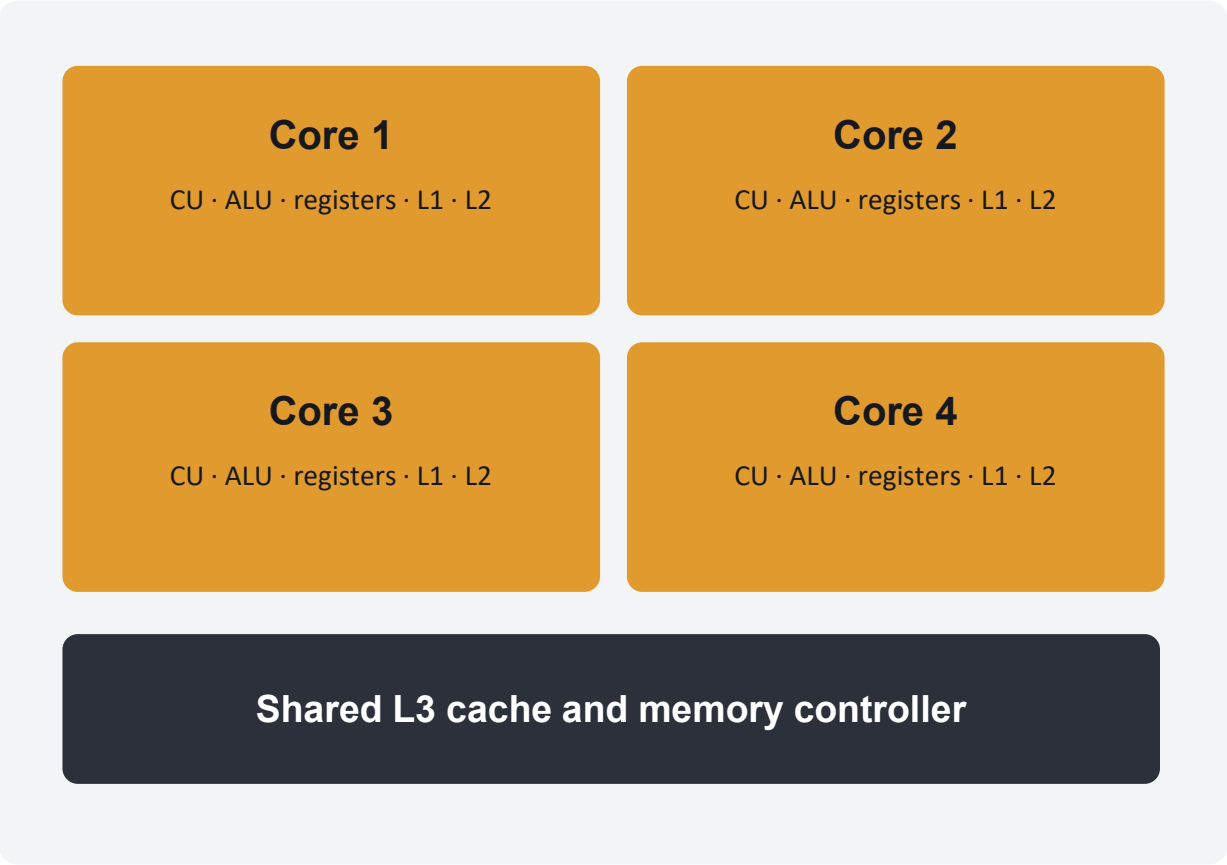
5

Modern processors

Multi-core, multithreading, system-on-chip, and two competing philosophies.



Multi-core in practice



- Each core is a complete processor with its own control unit, ALU, registers, L1 and L2 caches
- Cores share the larger caches and the path to main memory
- The operating system decides which core runs which process
- Real speed-up depends entirely on whether the work can be divided
- Some designs mix core types: a few fast cores for demanding work, several efficient cores for background tasks

This is where the extra transistors have gone since clock speeds stopped rising.

Getting more out of each core



Simultaneous multithreading

One physical core presents itself as two logical processors. When one thread stalls waiting for memory, the other uses the idle hardware. Intel calls this Hyper-Threading.



Branch prediction

The processor guesses which way a conditional jump will go and fetches ahead on that assumption. Modern predictors are right well over ninety per cent of the time.



Out-of-order execution

If the next instruction is waiting on memory, the processor runs a later independent instruction first, then puts the results back in order.



Vector instructions

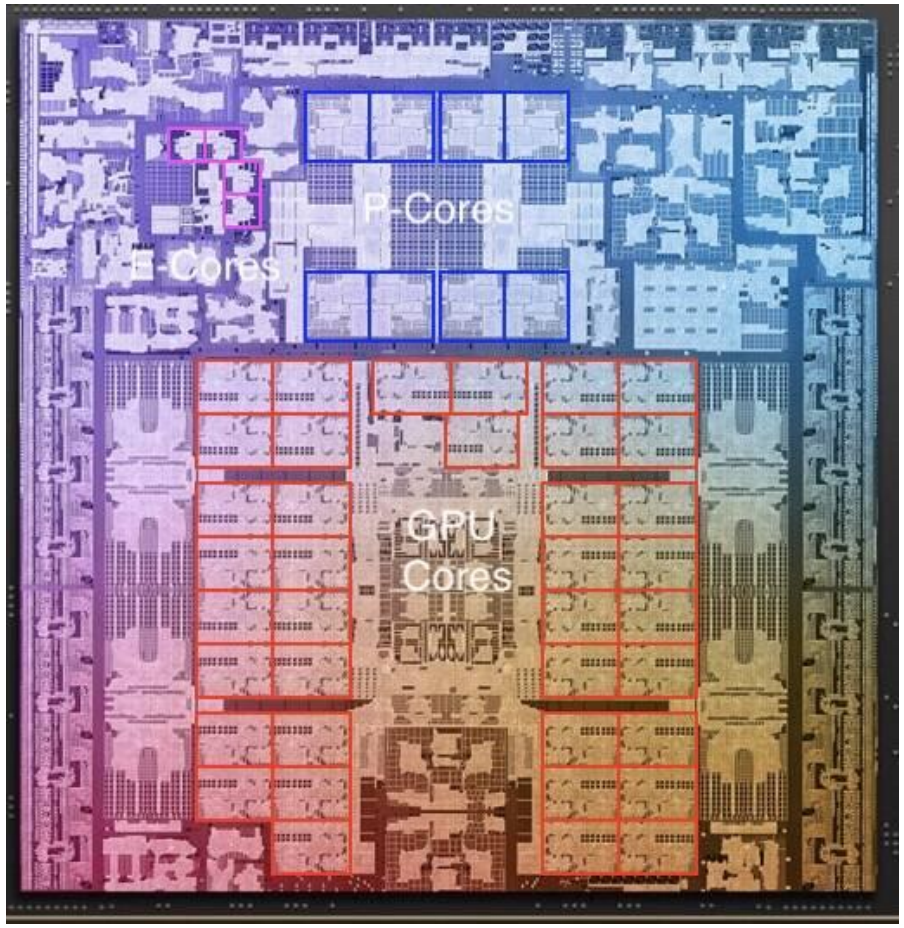
A single instruction applies the same operation to several values at once — valuable for graphics, audio and machine learning.

All four exist for one reason: keeping the execution units busy while memory catches up.

System-on-Chip (SoC)

- A whole computer on one piece of silicon, not just a processor
- CPU cores, graphics, memory controller, signal processors and often a neural accelerator, all on one die
- Shorter distances mean lower latency and much lower power consumption
- This is what is inside every phone, tablet and smart device — and increasingly inside laptops
- The trade-off: almost nothing can be upgraded or replaced separately

The boundary between ‘the processor’ and ‘the computer’ has largely dissolved.



Apple M-series SoC

Two philosophies: RISC and CISC

RISC

Reduced instruction set computer

- Fewer, simpler instructions
- Each takes about the same time — usually one cycle
- Fixed instruction length, easy to pipeline
- More instructions needed per task, so programs are longer
- For example, RISC would need LW, ADD and SW to add a register value directly to a value stored in memory
- Typically, lower power. Used by ARM: phones, tablets, Apple silicon

CISC

Complex instruction set computer

- Many instructions, some doing a great deal of work
- Instructions vary in length and in cycles taken
- Harder to pipeline, more complex to decode
- Fewer instructions needed per task, so programs are shorter
- For example, CISC only needs one ADD instruction to add a register value directly to a value stored in memory
- Used by x86: most desktops, laptops and servers

In practice the distinction has blurred:
modern x86 processors decode complex instructions into simple internal operations.

6

Data representation

Bits, bytes, number bases, and how a machine stores a negative number.



Bits and bytes

Bit

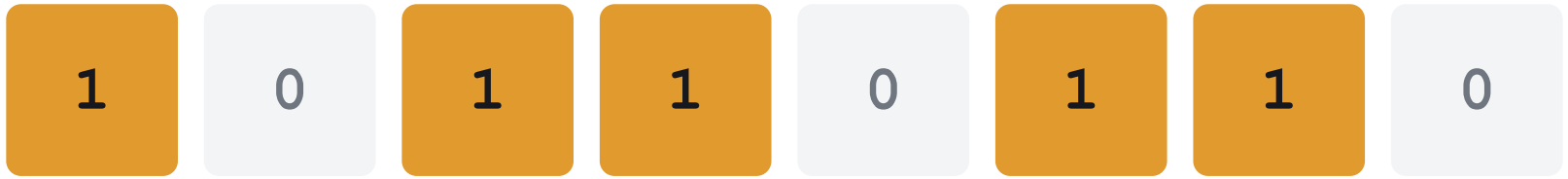
binary digit

- The on or off state of an electric current
- The basic unit of information, written as 0 or 1
- One bit has two possible values

Byte

eight bits

- Eight bits grouped together
- Enough to represent one character of text
- $2^8 = 256$ different combinations



8 bits = 1 byte

Everything — text, images, sound, video, this slide — is stored as patterns of bits. Only the interpretation differs.

Units of storage and transfer

STORAGE — BYTES

kilobyte	**KB**	10^3 = 1,000 bytes
megabyte	**MB**	10^6 bytes
gigabyte	**GB**	10^9 bytes
terabyte	**TB**	10^{12} bytes

TRANSFER RATES — BITS PER SECOND

kilobit	**kb**	1,000 bits
megabit	**Mb**	1,000,000 bits
gigabit	**Gb**	1,000,000,000 bits

A 100 Mbps connection delivers about 12.5 MB per second — divide by eight.

A COMMON SOURCE OF CONFUSION

Storage is sold in powers of ten; memory is addressed in powers of two. 1 KiB (kibibyte) = 2^{10} = 1,024 bytes, and 1 GiB = 2^{30} = 1,073,741,824 bytes. This is why a “1 TB” drive shows as about 931 GB in Windows.

Three number systems

Decimal

base 10

0 1 2 3 4 5 6 7 8 9

What people count in — ten fingers

Binary

base 2

0 1

What the machine stores — on and off

Hexadecimal

base 16

0-9 A B C D E F

A shorthand for people reading binary

THE SAME VALUE, THREE WAYS

Decimal	Binary	Hexadecimal
182	1011 0110	B6

Binary to decimal

Write the place values above the bits, then add up the columns that contain a 1.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
1	0	1	1	0	1	1	0
128	—	32	16	—	4	2	—

$$128 + 32 + 16 + 4 + 2 = 182$$

Your turn: what is 1100 1010 in decimal?

(answer on the practice slide)

Decimal to binary

METHOD 1 — SUBTRACT PLACE VALUES

Convert 202

128	fits in 202	1	leaves 74
64	fits in 74	1	leaves 10
32	too big	0	still 10
16	too big	0	still 10
8	fits in 10	1	leaves 2
4	too big	0	still 2
2	fits in 2	1	leaves 0
1	too big	0	done

202 = 1100 1010

METHOD 2 — DIVIDE BY TWO

Divide repeatedly, keep the remainders, then read them bottom to top.

202 ÷ 2	= 101	**r 0**
101 ÷ 2	= 50	**r 1**
50 ÷ 2	= 25	**r 0**
25 ÷ 2	= 12	**r 1**
12 ÷ 2	= 6	**r 0**
6 ÷ 2	= 3	**r 0**
3 ÷ 2	= 1	**r 1**
1 ÷ 2	= 0	**r 1**

↑ *read upwards*

1100 1010

Hexadecimal, and why it exists

THE PROBLEM

`1101 0110 1011 0110`

Binary is what the machine uses, but it is long, repetitive, and very easy to miscopy.

THE FIX

`D6B6`

Sixteen is 2^4, so one hex digit represents exactly four bits. Same value, a quarter of the characters.

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0000	0001	0010	0011	0100	0101	0110	0111
8	**9**	**A**	**B**	**C**	**D**	**E**	**F**
1000	1001	1010	1011	1100	1101	1110	1111

A to F carry the values ten to fifteen — a single symbol is needed for each.

Converting between binary and hex

Group the bits into fours from the right, then convert each group on its own.

1011

↓

8+0+2+1 = 11

B

0110

↓

0+4+2+0 = 6

6

1011 0110 = B6

Going the other way: expand each hex digit into its four bits.

`2F` → `0010 1111`

`C4` → `1100 0100`

Your turn

Two minutes. Work with the person next to you.

1 **1100 1010 in decimal?**

ANSWER

202

2 **1100 1010 in hex?**

ANSWER

CA

3 **75 in binary?**

ANSWER

0100 1011

4 **3F in binary?**

ANSWER

0011 1111

5 **1111 1111 in decimal?**

ANSWER

255

6 **Largest value in one byte?**

ANSWER

255 $(2^8 - 1)$

How do you store a negative number?

There is no minus sign in a register. There are only bits — so the sign must be encoded in the bits themselves.

ATTEMPT: SIGN AND MAGNITUDE

Use the leftmost bit as the sign: 0 for positive, 1 for negative.

```
0000 0101   =   +5
1000 0101   =   −5
```

- Simple to read — but two patterns mean zero: +0 and −0
- And ordinary addition gives the wrong answer

WHY THAT SECOND POINT MATTERS

```
  0000 0101      +5
+ 1000 0101      −5
  ──────────
  1000 1010      −10
```

It should be zero. So, the hardware would need a separate subtractor as well as an adder — more circuitry, more cost, more heat.

One’s complement

To negate a number, invert every bit. That is the whole rule.

+5	`0000 0101`
invert every bit	↓
−5	`1111 1010`

BETTER THAN SIGN AND MAGNITUDE

- The leftmost bit still shows the sign
- Addition nearly works — with an end-around carry added back in

STILL NOT GOOD ENOUGH

- Two representations of zero remain: 0000 0000 and 1111 1111
- The extra carry step costs hardware and time
- Range in eight bits: −127 to +127

Two’s complement

Invert every bit, then add one. This is what essentially every processor actually uses.

Start with +5	`0000 0101`
Step 1 — invert every bit	`1111 1010`
Step 2 — add one	`1111 1011`
Result: −5	`1111 1011`

WHY IT WINS

Only one zero, and ordinary addition just works — no separate subtractor needed.

RANGE IN EIGHT BITS

−128 to +127 — one more negative value than positive, because zero takes a positive slot.

Two’s complement arithmetic

SUBTRACTION BECOMES ADDITION

9 − 5 is computed as 9 + (−5)

```
   0000 1001      9
 + 1111 1011     −5
   ──────────
 1 0000 0100      4
```

Discard the carry out of the top — the answer is 4.

OVERFLOW — WHEN IT GOES WRONG

100 + 50 in eight signed bits

```
   0110 0100    100
 + 0011 0010     50
   ──────────
   1001 0110   −106
```

150 does not fit in −128…+127, so the result wraps into a negative number.

THIS IS A REAL CLASS OF BUG

Adding two positive numbers and getting a negative one means overflow has occurred. The processor sets an overflow flag; whether anyone checks it is up to the programmer. Silent integer overflow has caused everything from arcade-game score glitches to the loss of the Ariane 5 launcher in 1996.



Representing characters

Numbers are one thing. Text needs an agreed code that maps each character to a number.

ASCII

7 bits, usually stored in 8

128 characters: English letters, digits, punctuation and control codes. The foundation everything else builds on.

EBCDIC

8 bits

An IBM mainframe encoding, incompatible with ASCII. Still encountered in legacy financial systems.

Unicode

1-4 bytes in UTF-8

Over 1.1 million code points covering the writing systems of the world, plus symbols and emoji. The modern standard.

THE LETTER A IN ASCII

Decimal	Binary	Hexadecimal
65	0100 0001	41

The same byte, four meanings

Nothing in the bits themselves says which of these is intended. Only the context does.

0100 0001

As an unsigned number

65

As a two’s complement number

+65

As an ASCII character

A

As a machine instruction

whatever opcode 0x41 means on this CPU

This is the stored-program idea from Lecture 1, seen from the other end.

Summary

1. The CPU fetches, decodes and executes instructions — nothing more, but billions of times a second.
2. Inside it: a control unit that directs, an ALU that calculates, a clock that synchronises, buses that carry, and registers that hold.
3. The fetch-decode-execute cycle uses the PC, MAR, MDR, CIR and accumulator in a fixed sequence.
4. Performance comes from clock speed, cores, cache, width and pipelining together — never from one number alone.
5. Modern processors are multi-core systems-on-chip, using SMT, prediction and out-of-order execution to stay busy.
6. All data is bits. Hex is shorthand for binary, and two’s complement is how negatives are stored.

References and further reading

MATERIAL ADAPTED FOR THESE SLIDES

- Daley, B. Computers Are Your Future 2006, Complete Edition. Prentice Hall, 2006.
- Oak National Academy / Raspberry Pi Foundation, CPU components teaching materials — Open Government Licence v3.0

READING FOR THIS TOPIC

- Williams, Computer Systems Architecture — the module core text
- Patterson & Hennessy, Computer Organization and Design — chapters on the processor and arithmetic

BEFORE NEXT WEEK

Practise conversions until they are automatic: decimal to binary, binary to hex, and two’s complement negation of any eight-bit value. Everything in the assembly labs assumes you can do these without thinking.